

The Browser as an Embodied AI Substrate: Accessible Physical Simulation for Trajectory Data Generation

Greatness Okorie

Bells University of Technology

Ota, Ogun State, Nigeria

Okorie.greatness@bellsuniversity.edu.ng

ORCID: 0009-0007-0940-123X

Abstract

Large language models scaled rapidly because internet-scale text data was already broadly available. Embodied artificial intelligence faces a different constraint. Physical interaction data remains expensive to collect and is often gated by specialized simulation stacks, complex installation pipelines, and high-performance local hardware. This creates barriers for independent researchers, educational settings, and institutions without access to graphics processing unit clusters or robotics laboratories.

In this paper we examine whether browser-native physical simulation can function as an accessible substrate for generating structured embodied trajectory data. We present an architecture that combines a physics engine compiled to WebAssembly, real-time browser rendering, client-side agent control, external model inference, episodic memory, and multi-format dataset export. The architecture is designed to run the core physics and rendering loop without requiring a local graphics processing unit. We instantiate the architecture in an open platform named Synthia that supports an articulated humanoid agent, records observation-action-reward trajectories, and exports them in research-compatible formats including JSONL, Parquet, HDF5, and LeRobot-compatible schemas. We report preliminary feasibility observations across devices and browsers, describe the data schema, and discuss accessibility advantages relative to traditional local simulation stacks.

We argue that web-native simulation is not merely an engineering convenience. It represents a potentially important route toward democratizing the generation of embodied experience data and enabling broader participation in embodied AI research. Significant limitations remain, including current embodiment scope, performance ceilings, and the absence of closed-loop policy improvement, all of which are stated explicitly.

1. Introduction

The rapid advancement of Large Language Models was predicated on the existence of a public, internet-scale corpus of text data. This open substrate allowed researchers worldwide to train, evaluate, and iterate on increasingly capable systems. In stark contrast, the field of Embodied Artificial Intelligence faces a significant bottleneck in the form of data scarcity and access inequality. While language models could leverage existing textual information, embodied agents require physical interaction data. This data must be either collected from expensive hardware or simulated in computationally intensive environments.

This reliance on specialized infrastructure has created a formidable barrier to entry. It concentrates research within well-funded laboratories and large corporations while marginalizing independent researchers, students, and institutions in resource-constrained regions. Traditional high-fidelity simulators often necessitate powerful workstation-class graphics processing units, significant video memory, and specific operating systems. Even more accessible simulators require complex installation processes involving specific dependencies and environment management, which can be prohibitive for non-specialists.

In this paper, we examine whether browser-native physical simulation can function as an accessible substrate for generating structured embodied trajectory data. We present a zero-install architecture that combines a physics engine compiled to WebAssembly, real-time browser rendering, client-side agent control, external model inference, episodic memory, and multi-format dataset export. We instantiate this architecture in an open platform named Synthia to validate its feasibility.

The core contributions of this work are as follows:

1. A zero-install architecture for browser-native physical simulation that decouples the core agent loop from local hardware dependencies.
2. An open implementation named Synthia demonstrating the feasibility of client-side trajectory generation for complex articulated agents.
3. A structured, multi-format dataset export pipeline compatible with modern robotics frameworks.
4. Preliminary accessibility and feasibility observations illustrating the trade-offs of browser-based execution.

This work is a systems and accessibility contribution. It does not propose a new learning algorithm, nor does it claim to present a fully autonomous self-evolving agent. Rather, it provides the foundational infrastructure required to make embodied data generation accessible to a broader research community.

2. Motivation: The Embodied Data Bottleneck

Text data is distributed by default. The internet provided a naturally occurring, globally accessible corpus that served as the foundation for modern natural language processing. Physical interaction data possesses no such natural distribution. Collecting real-world robotic data requires expensive hardware, controlled environments, and significant human labor for teleoperation or annotation.

Simulation is the primary scalable substitute for real-world data collection. However, the simulation ecosystem is currently infrastructure-heavy. High-fidelity simulators provide excellent rendering and physics fidelity but demand substantial computational resources. They frequently require specific Linux distributions, complex driver configurations, and high-end dedicated graphics hardware. This creates an infrastructure gatekeeping effect. Researchers in the Global South, students at underfunded universities, and independent engineers are often excluded from the field simply because they cannot provision the necessary local compute environment.

If embodied artificial intelligence is to achieve the same breadth of participation as language modeling, the generation of physical experience must become as ordinary as browsing the web. The web browser is the most ubiquitous and standardized execution environment in the world. By shifting the computational burden of physical simulation from the local machine to the browser environment, we can transform the browser from a passive content viewer into an active execution platform for embodied research.

The central research question is therefore:

Can the browser become a viable open substrate for generating structured embodied trajectory data?

We answer this question by presenting an architecture that runs the core embodied loop in the browser and by using an open implementation to demonstrate feasibility.

3. Related Work

The development of embodied artificial intelligence relies heavily on simulation platforms and standardized datasets.

High-fidelity simulators such as Isaac Sim provide enterprise-grade physics and rendering but require significant local hardware investment [1]. MuJoCo remains a standard for contact-rich rigid-body dynamics and has been widely adopted for reinforcement learning research [2]. PyBullet offers a more accessible, albeit lower-fidelity, alternative that runs on consumer hardware [3]. Habitat and PyRep have also been instrumental in standardizing environments for navigation and manipulation tasks [4, 5]. While these tools are powerful, they universally require local installation, native compilation, and environment management.

The robotics community has recently made significant strides in standardizing datasets. The Open X-Embodiment collaboration demonstrated the value of aggregating diverse robotic trajectories [6]. DROID provides large-scale real-world manipulation data [7]. Crucially, the LeRobot initiative has

established a unified structure and open-source library for organizing multi-modal robotics data, making it a de facto standard for modern imitation learning pipelines [8].

Concurrent efforts have explored WebAssembly for browser-based physics. Various open-source projects have successfully compiled physics engines to run in the browser for visualization or simple interactive demos. However, these efforts typically focus on rendering or basic physical playback. The present work differentiates itself by targeting a complete client-side agent loop, integrating multi-tiered memory, external inference-driven action selection, and structured multi-format trajectory export designed specifically for accessible robotics research and downstream policy training.

4. Browser-Native Embodied Simulation Architecture

The architecture presented is designed to realize the goal of a zero-install, hardware-agnostic embodied simulation environment. The core design principles are accessibility, modularity, and data-centricity. The architecture eschews heavy local dependencies in favor of client-side execution entirely within the browser.

4.1 Physics Layer

The foundational component is the physics engine (Mujoco), which is compiled to WebAssembly. By compiling high-performance C-based physics code to WebAssembly, the core simulation logic becomes portable and runs directly in the browser without requiring native compilation. This layer handles complex calculations for joint actuation, contact forces, and rigid-body dynamics in real time. This choice provides a direct link to the academic robotics community, as standard physics engines are widely recognized tools, lending credibility to the simulation fidelity.

4.2 Rendering Layer

For visualization, the architecture leverages standard JavaScript libraries for creating and displaying three-dimensional graphics in web browsers using WebGL. This layer provides a high-level interface for managing scenes, cameras, lights, and meshes. Its integration with the WebAssembly-compiled physics engine allows for the real-time rendering of the simulated environment and articulated agents, providing essential perceptual data for the agent loop.

4.3 Agent Loop

The agent loop is responsible for perception, action selection, action execution, and memory recording. Observations are assembled from the rendered scene, physics state, proprioceptive signals, episode metadata, and optional multimodal inputs. Action selection is supplied by external AI inference: the client constructs a perception payload from the current physics state and visual observations, streams a response from a user-configured language model provider through an application programming

interface, parses structured joint commands, and applies them to the physics body. When no external provider is configured, the system can use scripted controllers for baseline behavior and system testing. Each resulting transition is recorded as a structured trajectory point containing observation, action, outcome, reward, goal, and termination information.

4.4 Memory System

To support complex behavior and richer trajectory metadata, the architecture incorporates a multi-tiered memory system. This includes working memory for short-term state tracking, episodic memory to store entire sequences of experiences from individual sessions, and long-term memory for persistent goals. The system is designed to support semantic retrieval, though initial instantiations may rely on deterministic retrieval mechanisms while full client-side embedding models are optimized for browser constraints.

The practical value of the memory system is that it allows exported trajectories to contain more than low-level control signals. Memories can include goals, beliefs, termination reasons, session identifiers, and identity versions. This metadata is useful for downstream dataset filtering, imitation learning, behavioral analysis, and future curriculum construction.

4.5 Identity Modification

The architecture includes a primitive form of introspection via limited identity self-modification. The mechanism permits limited, rate-limited, logged parameter updates to agent settings such as name, stated goals, or behavioral preferences. All modifications are strictly logged. This mechanism is framed as an early primitive that may support future self-evolving agent architectures, rather than a claim of fully closed-loop self-evolution.

4.6 Dataset Export Pipeline

The architecture culminates in a robust dataset export pipeline. This component serializes the recorded trajectory data into standardized formats, including JSONL, CSV, Parquet, and HDF5. Critically, it supports export to the LeRobot format, ensuring that the data generated is compatible with the tools and workflows of the wider robotics research community [8].

The export pipeline is one of the central practical contributions of this work. Accessibility is only useful if the generated data can enter existing research workflows. By supporting common formats, the architecture allows browser-generated trajectories to be inspected, stored, shared, and used for downstream learning experiments.

5. Implementation and Instantiation in Synthia

The architectural principles are instantiated in an open-source platform named Synthia, which serves as the empirical apparatus for this research [9].

The platform is publicly available at <https://runsynthia.online> and the source code is available at <https://github.com/Greatness0123/synthia>

Synthia is not the subject of the paper but rather the proof-of-concept that validates the viability of a browser-native substrate for embodied AI data generation.

The platform currently focuses on a single, complex embodiment consisting of an articulated humanoid agent with approximately 80 degrees of freedom. This choice of embodiment is strategically significant, as humanoid robots represent a frontier in robotics research, facing challenges related to dynamic balance and whole-body coordination. The humanoid is equipped with a layered balance controller stack to maintain stability, a fundamental prerequisite for generating useful locomotion data.

The user interacts with the platform through a standard web browser, eliminating any need for local installation. Upon loading the page, the browser asynchronously fetches the necessary assets, including the rendering library, the pre-compiled physics module, the agent model definition, and the control script. Once loaded, the simulation begins immediately.

The platform supports multi-agent scenarios within a shared world, allowing for the study of social interactions. A dedicated training mode provides a structured environment for data collection, automatically terminating episodes based on predefined success or failure criteria. Crucially, every step of the agent interaction is recorded. Each data point contains proprioceptive state, goal specifications, session identifiers, and termination events, transforming simple sensor logs into structured, semantically meaningful trajectories.

Synthia is therefore used as evidence that the proposed architecture can operate in a realistic browser setting. The broader claim is not that Synthia itself is the contribution, but that the architecture it instantiates is feasible.

6. Preliminary Evaluation

A central claim of this work is the accessibility advantage of the browser-native approach. To substantiate this, preliminary feasibility observations were collected. It is crucial to acknowledge upfront that browser-based simulation will not match the throughput of dedicated, graphics-accelerated clusters used for massive parallel training. Its value proposition lies in drastically lowered entry barriers.

The evaluation in this draft is a feasibility demonstration rather than a controlled benchmark. Quantitative results for render rate, physics-step rate, latency, memory usage, and load time must be collected under a fixed protocol before stronger performance claims are made.

6.1 Performance Feasibility

The performance evaluation in this subsection is a preliminary feasibility study rather than a controlled benchmark. The objective is to show that the core physics, rendering, and agent loop can operate on ordinary consumer hardware without a dedicated graphics processing unit.

The measurements reported in Table 1 are preliminary feasibility observations collected during manual testing. They are not intended to establish throughput parity with high-performance local simulation stacks or large-scale accelerated simulators.

The intended benchmark protocol is as follows:

Device Class	Browser	Render FPS	Physics Steps per Second	Agent Loop Latency	Initial Load Time	Peak Memory
Low-end laptop	Chrome	28–35 FPS	40–50 steps/s	18–25 ms	4.2 s	310 MB
Mid-tier laptop	Chrome	55–60 FPS	58–60 steps/s	8–12 ms	2.1 s	420 MB
Desktop computer	Chrome	60 FPS (capped)	60 steps/s	3–6 ms	1.3 s	490 MB
Mid-tier laptop	Firefox or Edge	50–58 FPS	55–60 steps/s	10–15 ms	2.4 s	445 MB

Table 1: Preliminary feasibility observations for the core browser-native loop.

Note: Low-end laptop tested on Intel i5-10210U (Integrated UHD 620); Mid-tier laptop tested on Apple M1 (8-core CPU/GPU); Desktop tested on Ryzen 7 5800X / RTX 3070.

6.2 Accessibility Comparison

The accessibility advantage is best understood through a qualitative comparison of the onboarding process and resource requirements.

Table 2 contrasts the browser-native approach with a typical local installation of a high-fidelity simulator.

Table 2: Accessibility and Infrastructure Comparison

Dimension	Traditional Local Simulator	Browser-Native Approach
Installation	Complex; requires package managers, specific drivers, and manual dependency resolution.	Minimal; requires no installation. Launch a uniform resource locator in a modern browser.
Operating System	Primarily restricted to specific Linux distributions.	Cross-platform; functions on any operating system supporting a compliant browser.

Dimension	Traditional Local Simulator	Browser-Native Approach
Hardware Requirement	Often requires a powerful, dedicated graphics processing unit.	Not required for the core physics and rendering loop. Performance depends on the central processing unit.
Onboarding Time	High; can take hours or days to configure the environment correctly.	Very Low; the environment is ready to use upon page load.
Dataset Export	Varies by platform and often requires custom scripting.	Built-in export to JSONL, CSV, Parquet, HDF5, and LeRobot-compatible schemas.

This comparison does not claim that the browser-native approach is superior in throughput or scalability. It claims that the browser-native approach materially lowers the barrier to first use, first data generation, and first experimentation.

6.3 Data Utility Demonstration and Ecosystem Compatibility

To validate the research utility of exported trajectories, we performed an end-to-end format integration test with mainstream robotics tooling. Trajectories recorded during a 50-episode humanoid balancing session were serialized entirely client-side using Synthia's built-in exporter and saved in the Hugging Face LeRobot dataset format (v2.0 schema).

The exported dataset was directly loaded into an isolated Python 3.10 environment using the official lerobot library. The dataset passed automated validation with zero structural modifications or field casting required:

Feature Mapping: Proprioceptive states (observation.state), visual frames (observation.image), and motor targets (action) successfully mapped into PyTorch-compatible torch.Tensor objects with correct tensor shapes ($[N, 80]$ for joint states).

Metadata Preservation: Episode boundaries, session identifiers, and termination flags were properly indexed in the dataset manifest, confirming full compatibility with standard imitation learning dataloaders (torch.utils.data.DataLoader).

Table 3: Example Trajectory Schema

Field	Description
session_id	Identifier for the browser session.
episode_id	Identifier for the current episode.
agent_id	Identifier for the agent in a multi-agent world.
timestamp	Time of the recorded step.
observation.proprioception	Joint positions, velocities, and related body state.
observation.perception_summary	Structured summary of perceived scene information.

Field	Description
observation.vision_summary	Optional summary from external vision inference.
action.joint_commands	Structured command values applied to the physics body.
reward	Scalar reward emitted for the step.
goal	Current task or goal specification.
termination_reason	Reason the episode ended, if applicable.
memory_tier	Memory tier associated with the record.
identity_version	Version identifier for agent identity parameters.

This schema supports imitation learning, behavioral cloning, dataset filtering, and later reinforcement-learning pipelines. It also supports richer analysis because the trajectory includes goal context, termination context, and memory metadata.

7. Discussion: Accessibility as a Research Accelerator

This work argues that the browser can serve as a viable, accessible substrate for generating the physical interaction data that is currently a major bottleneck in embodied artificial intelligence research. The proposed architecture successfully decouples high-fidelity physical simulation from the requirement of specialized local hardware.

The primary contribution is the demonstration that a complete, end-to-end data generation pipeline can be executed client-side. This represents a significant step toward democratizing the field, lowering the financial and infrastructural barriers that have historically excluded independent researchers and laboratories in resource-constrained regions. By enabling a distributed model of data generation, this approach holds the potential to create large-scale, diverse, and publicly accessible datasets analogous to the role of web-crawled text in natural language processing. Such datasets would be invaluable for training more robust and generalizable policies.

Research democratization is the central implication. If embodied data generation becomes accessible through ordinary browsers, the field may benefit from a broader distribution of data generation, experimentation, and evaluation. Independent researchers could generate local datasets without purchasing high-end workstations. Laboratories with limited funding could participate in embodied AI research using existing laptop hardware. Educational institutions could introduce students to embodied agents without dedicating an entire course to installation troubleshooting.

Beyond research democratization, this architecture offers significant educational upside. Students and educators can instantly access complex physical simulations for coursework and experimentation without spending lab sessions troubleshooting driver installations. This immediate reproducibility fosters a more inclusive and rapid learning environment.

The browser-native approach also enables reproducible public demonstrations. A research artifact can be shared as a URL rather than as a compressed archive containing code, models, and installation instructions. This lowers the cost of validation and makes it easier for reviewers, students, and collaborators to inspect the system directly.

8. Limitations

It is imperative to be transparent about the current limitations of the system and the evaluation.

First, the platform currently supports only a single primary embodiment. While the architecture is modular, extending support to other robot types remains a key area for future development.

Second, the system does not yet implement a closed-loop learning or policy improvement mechanism. The current implementation includes primitive identity parameter updates and memory persistence, but it does not improve its own control policies.

Third, while performance is sufficient for preliminary real-time interaction and data logging on consumer hardware, it does not compete with the throughput of large-scale accelerated simulators. The current evaluation does not provide a controlled benchmark, and stronger performance claims require formal measurement.

Fourth, external inference may depend on remote model providers. While the core physics and rendering loop can run locally in the browser, action selection through large language models may require network access and external inference services. This distinction should be stated clearly.

9. Future Work

Looking forward, several promising avenues for development exist. A critical next step is to implement closed-loop learning pipelines, enabling agents to learn directly from the data they generate within the browser environment. Integrating the platform with interactive imitation learning frameworks would transform it from a static data generator into a dynamic Reinforcement learning environment.

Another major direction is expanding the platform extensibility by adding support for alternative physics engines compiled to WebAssembly. This would allow researchers to test the cross-engine generalization of policies. Technologically, migrating to WebGPU, the successor to WebGL, could unlock significant performance gains for both rendering and physics computations, further improving the fidelity and speed of the simulation.

10. Conclusion

Embodied artificial intelligence cannot rely solely on text-scale data because embodied experience is not yet publicly available at scale. Browser-native simulation offers a promising route toward lowering the barrier to generating such experience. By combining WebAssembly physics, browser rendering, structured memory, external inference, and standardized dataset export, the architecture presented in this work demonstrates that a useful embodied agent loop can run without local specialized hardware.

The open implementation, Synthia, provides preliminary evidence that this architecture is feasible. It supports an articulated humanoid agent, records structured observation-action-reward trajectories, and exports them in research-compatible formats. Significant limitations remain regarding embodiment scope, benchmarking, and closed-loop learning. Nonetheless, this work suggests that the browser can serve as an accessible substrate for embodied AI research, potentially democratizing the generation of physical agent data for the global research community.

Code and Data Availability.

The browser-native architecture described in this paper is implemented in Synthia, an open-source platform. The live platform is available at <https://runsynthia.online> and the source code is available at <https://github.com/Greatness0123/synthia>. Exported dataset schemas and example trajectory files are included in the repository.

Acknowledgments: The author thanks the open-source robotics community for the development of MuJoCo, LeRobot, and related tools that made this work possible.

References

- [1] NVIDIA. Isaac Sim: Robotics simulation and synthetic data generation. 2023.
- [2] E. Todorov, T. Erez, and Y. Tassa. MuJoCo: A physics engine for model-based control. In IEEE/RSJ International Conference on Intelligent Robots and Systems, 2012.
- [3] E. Coumans and R. Bai. PyBullet: A Python module for physics simulation for games, robotics, and machine learning. 2016.
- [4] M. Savva et al. Habitat: A platform for embodied AI research. In IEEE/CVF International Conference on Computer Vision, 2019.
- [5] S. James et al. PyRep: Bringing the V-REP simulator to researchers. In Conference on Robot Learning, 2019.
- [6] Open X-Embodiment Collaboration. Open X-Embodiment: Robotic learning datasets and RT-X models. arXiv preprint arXiv:2310.08864, 2023.

- [7] A. Khazatsky et al. DROID: A large-scale in-the-wild robot manipulation dataset. arXiv preprint arXiv:2403.12945, 2024.
- [8] Hugging Face. LeRobot: State-of-the-art machine learning for real-world robotics. 2024.
- [9] G. Okorie. Synthia: A browser-native embodied AI platform for accessible trajectory data generation. Open-source software, 2026.
- [10] WebAssembly Community Group. WebAssembly Core Specification. W3C, 2024.
- [11] T. Zhao et al. Learning fine-grained bimanual manipulation with low-cost hardware. In Robotics: Science and Systems, 2023.
- [12] A. Brohan et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. In Conference on Robot Learning, 2023.
- [13] S. Levine et al. End-to-end training of deep visuomotor policies. Journal of Machine Learning Research, 2016.
- [14] J. Schulman et al. Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347, 2017.
- [15] G. Brockman et al. OpenAI Gym. arXiv preprint arXiv:1606.01540, 2016.
- [16] Y. Duan et al. Benchmarking deep reinforcement learning for continuous control. In International Conference on Machine Learning, 2016.
- [17] F. Xia et al. Interactive language: Talking to robots in real time. IEEE Robotics and Automation Letters, 2022.
- [18] M. Ahn et al. Do as I can, not as I say: Grounding language in robotic affordances. In Conference on Robot Learning, 2022.
- [19] Apache Software Foundation. Apache Parquet. 2024.
- [20] The HDF Group. HDF5. 2024.